



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 4 – Templates and Exception Handling
Topic – 4.3 typename and export keywords in Template

Prof. V.N.Nirgude
Assistant Professor

E-mail :

nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215



typename keyword

- In the template parameter list of a template declaration, typename can be used as an alternative to class to declare type template parameters(since C++17).
- Inside a declaration or a definition of a template, typename can be used to declare that independent qualified name is a type.
- There are two uses for the template and typename keywords in C++.
- One of them is fairly well known amongst programmers: to define templates.
- The other use is: to specify that an expression refers to a template function or a type.



Using the template and typename keywords to define Templates

- The template and typename keywords are routinely used to define templates.

- Example:

```
template <typename T>
```

```
bool isPositive(T x)
```

```
{
```

```
    return 1;
```

```
}
```

- We could just as well have written `template <class T>`; the keywords `typename` and `class` have the same meaning in this context.



Export Template/Use of export keyword

- The C++ standard defines the concept of "export templates".
- These are template classes and functions which are only declared wherever they are used (ie. usually in a header file), while implemented only once in some compilation unit.
- Exported templates are not supported with g++ 3.3 or earlier. You need g++ 3.4 or newer to use this feature.
- Exported templates are templates declared with the export keyword.
- An exported template is unique because its definition does not need to be present in a translation unit that uses that template.

While creating exported template here, it consists of three files:

- The file *plus.hpp* contains the declaration of an exported function template `plus()`.
- The file *plus.cpp* contains the definition of template function `plus()`.
- The file *test.cpp* includes and defines a `main()` function that uses `plus()`.



A simple program using exported templates

plus.hpp:

```
#ifndef PLUS_HPP_INCLUDED
#define PLUS_HPP_INCLUDED
export template<typename T>
T plus(const T& lhs, const T& rhs);
#endif
```

plus.cpp:

```
#include "plus.hpp"
template<typename T>
T plus(const T& lhs, const T& rhs)
```

```
{
    return rhs + lhs;
}
```

test.cpp:

```
#include <iostream>
#include "plus.hpp"
int main()
{
    cout << "2 + 2 = " << plus(2, 2) ;
}
```

Thank You..